Caching in Python Applications

lru_cache, cachetools, Redis from async code, and good keys

Phase 3 videos on YouTube

> **In one line:** functools.lru_cache suits pure functions in one process but has no TTL and is per worker; cachetools adds TTL and size-bounded caches; with many workers and pods, shared or changing data belongs in Redis via redis.asyncio; choose serialization deliberately, and build keys from a namespace, a version and every input that changes the result.

1 Fast in the test, broken in production

You add `@functools.lru_cache` to a slow function and it gets a hundred times faster in your test. Then you deploy with eight Gunicorn workers on four pods, and the hit ratio is terrible, memory grows, and users see stale data. Python caching has a few traps.

2 functools.lru_cache

It wraps a function and remembers results by arguments, with a `maxsize` (and `@functools.cache` is the unbounded version). Perfect for **pure functions** with hashable arguments: parsing a config, computing a small lookup. But it has **no TTL**: cached values never expire on their own. Use `f.cache_clear()` to reset, and `f.cache_info()` to see hits and misses. On methods it keeps `self` alive, so prefer module-level functions.

3 cachetools, and async code

The **cachetools** library adds control: `TTLCache(maxsize=10_000, ttl=60)` evicts entries after a time, `LRUCache` and `LFUCache` bound size, and the `@cached(cache)` decorator wraps functions with any of them (pass a lock if threads share it).

For **async** code, remember that a plain `lru_cache` on an `async def` function caches the **coroutine object**, not the result: the second await fails. Use an async-aware cache (for example `async-lru` or `aiocache`), or cache the awaited result yourself.

4 The multi-worker trap

Each worker **process** has its own separate cache. With 8 workers on 4 pods that's **32 independent caches**. Each one warms up separately, so hit ratios are low; memory is duplicated 32 times; and invalidation only clears one worker's cache, so the other 31 keep serving stale values. For shared or changing data, use Redis.



8 Gunicorn workers × 4 pods = 32 separate caches: each warms alone, memory ×32, and cache_clear() in one worker leaves 31 still serving the old value

Figure 1. In-process caches are per worker process, not per service.

5 Redis from async code, and serialization

From FastAPI or other async code, use **redis.asyncio** (part of redis-py), so cache calls don't block the event loop, and reuse one client with its connection pool. Then think about **serialization**: values must become bytes. For big objects, serialization can cost more time than the Redis call itself.

format	speed / size	portable?	watch out
JSON	slower, larger	any language	no datetimes/bytes natively
msgpack	fast, compact	many languages	a schema is still on you
pickle	fast, any object	Python only	unsafe with untrusted data

Figure 2. Three common serializations and their trade-offs.

6 What goes into a cache key

Everything that changes the result, and nothing else.

- A **namespace** for the service and data type: `search:`.
- A **schema version**: `v2:`. Bump it when the format changes, and old entries are simply ignored until they expire.
- The **identifying inputs**: `tenant42:` plus a hash of the normalised query.
- The **user or tenant** whenever results differ per user, or you'll leak data between them.
- Nothing irrelevant, like a timestamp or a request ID, which makes every key unique and the hit ratio zero.

Example: `search:v2:tenant42:9f86d081…`.

WORKED EXAMPLE: AN EMBEDDING CACHE

A FastAPI service computes text embeddings with a model call that takes 200 ms. Before calling the model it normalises and hashes the text and looks up `embed:v1:<model>:<sha256>` in Redis with redis.asyncio. On a hit it deserialises the vector from msgpack in microseconds; on a miss it calls the model and stores the result with a one-day TTL. Repeat texts now cost about a millisecond instead of 200, and the model name in the key means switching models never returns old vectors.

```python
import hashlib, msgpack, redis.asyncio as redis
r = redis.Redis()

async def embed(text: str, model='text-emb-3'):
    norm = ' '.join(text.lower().split())
    key = f'embed:v1:{model}:' + hashlib.sha256(norm.encode()).hexdigest()
    if (hit := await r.get(key)) is not None:
        return msgpack.unpackb(hit)          # ~1 ms
    vec = await call_model(norm, model)      # ~200 ms
    await r.set(key, msgpack.packb(vec), ex=86_400)
    return vec
```

Figure 3. The embedding cache in a few lines: namespace, version, model and a content hash in the key.

COMMON MISTAKES

- `lru_cache` for data that changes, with no way to expire it across workers.
- `lru_cache` on an async function.
- Unpickling data from a cache other systems can write to: remote code execution.
- Keys without the tenant, so one customer sees another's results.

7 Key terms

lru_cache: per-process memoisation with LRU, no TTL

cachetools: TTLCache, LRUCache, LFUCache + @cached

redis.asyncio: non-blocking Redis client in redis-py

Serialization: turning objects into bytes

Key namespace: prefix naming the service and data type

Key version: bump to ignore old formats

8 Interview questions

Q1. What should a cache key contain? BASIC

A namespace, a version, and every input that changes the result, including the tenant or user when results differ, but nothing irrelevant like timestamps.

Q2. What are the pitfalls of lru_cache in multi-worker deployments? BASIC

Every process has its own cache, so hit ratios drop, memory is duplicated, there's no TTL, and invalidation only reaches one worker, so shared or changing data belongs in Redis.

Q3. Why not put lru_cache on an async function? BASIC

It caches the coroutine object; awaiting it a second time raises an error. Use an async-aware cache or cache the awaited result.

Q4. Design caching for an LLM-backed endpoint with repeated prompts. SCENARIO

Normalise the prompt, hash it with the model and parameters into a versioned key scoped to the tenant, store the response in Redis with a TTL, and coalesce concurrent identical requests.

Q5. When is pickle acceptable for cache values? TRICKY

Only when the cache is private to trusted Python services, versions are compatible, and nobody else can write to it; otherwise use JSON or msgpack.

Q6. Your cache hit ratio is 0% even though the same data is requested. What do you check?

TRICKY

The keys: something varying (a timestamp, request ID, unsorted parameters, different whitespace) is in the key, or each worker has its own cold in-process cache.

Next: SD-J-001 · Stateless Service Design